

Оценка производительности кластеризации

1. Метрики, требующие "истинных" меток (С учителем)

Эти метрики используются, когда у нас есть размеченные данные (например, для тестирования алгоритма). Они сравнивают полученные кластера с реальными классами.

Задача 1: Оценка качества кластеризации K-Means на ирисах Фишера

Условие: есть датасет ирисов (3 класса по 50 образцов). Мы применили K-Means, чтобы разбить данные на 3 кластера. Нужно оценить, насколько хорошо полученная кластеризация совпадает с реальными видами ирисов.

Программа:

```
import numpy as np

from sklearn import datasets, metrics
from sklearn.cluster import KMeans

# 1. Загружаем данные
iris = datasets.load_iris()
X, y_true = iris.data, iris.target

# 2. Применяем кластеризацию (K-Means, 3 кластера)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
y_pred = kmeans.fit_predict(X)

# 3. Оценка производительности
print("Оценка качества кластеризации:")

print(f"Скорректированный индекс Rand (ARI):  
{metrics.adjusted_rand_score(y_true, y_pred):.3f}")

print(f"Взаимная информация (V-measure): {metrics.v_measure_score(y_true,  
y_pred):.3f}")

print(f"Индекс Фаулкса-Мэллоуза (FMI):  
{metrics.fowlkes_mallows_score(y_true, y_pred):.3f}")

# Дополнительно: Матрица неточностей (Contingency Matrix)
print("\nМатрица неточностей (Истина vs Предсказание):")
print(metrics.cluster.contingency_matrix(y_true, y_pred))
```

Объяснение:

Оценка качества кластеризации:

Скорректированный индекс Rand (ARI): 0.730

Взаимная информация (V-measure): 0.758

Индекс Фаулкса-Мэллоуза (FMI): 0.821

Матрица неточностей (Истина vs Предсказание):

[[0 50 0]

[48 0 2]

[14 0 36]]

`adjusted_rand_score (ARI)`: Показывает сходство двух разбиений с поправкой на случайность. **Результат (~0.73)** означает хорошее, но не идеальное совпадение. Идеал = 1.0. ARI симметричен и является одной из самых популярных метрик.

`v_measure_score`: Это гармоническое среднее между *Однородностью* (все точки в кластере одного класса) и *Полнотой* (все точки класса попали в один кластер). **Результат (~0.75)** также говорит о хорошем качестве.

`fowlkes_mallows_score (FMI)`: Среднее геометрическое точности и полноты для пар точек. Чем ближе к 1, тем лучше.

`contingency_matrix`: Показывает, сколько объектов из каждого истинного класса попало в каждый предсказанный кластер.

Первая строка (класс 0): [0, 47, 3] — все 50 ирисов класса 0 разбились на кластеры 1 и 2, ни одного не попало в кластер 0.

Это объясняет, почему ARI не 1.0 — алгоритм перепутал метки кластеров. Но так как ARI учитывает только *разбиение*, а не названия кластеров, оценка все равно высокая.

2. Метрики, не требующие "истинных" меток (Без учителя)

Эти метрики используются, когда у нас нет правильных ответов. Они оценивают, насколько "хорошо" кластера отделены друг от друга и насколько они компактны.

Задача 2: Выбор лучшего числа кластеров K для K-Means

Условие: Мы не знаем, на сколько кластеров делятся наши данные (те же ирисы, но мы делаем вид, что меток классов у нас нет). Нужно выбрать оптимальное число кластеров K, используя внутренние метрики.

Программа:

```

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score, calinski_harabasz_score,
davies_bouldin_score

X, _ = datasets.load_iris(return_X_y=True) # Игнорируем истинные метки y
# Словари для хранения результатов

silhouette_scores = []

ch_scores = []

db_scores = []

K_range = range(2, 6) # Обычно проверяют K от 2 и выше

for k in K_range:

    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)

    labels = kmeans.fit_predict(X)

    # Расчет метрик

    sil_score = silhouette_score(X, labels)

    ch_score = calinski_harabasz_score(X, labels)

    db_score = davies_bouldin_score(X, labels)

    silhouette_scores.append(sil_score)

    ch_scores.append(ch_score)

    db_scores.append(db_score)

    print(f'K={k}: Silhouette={sil_score:.3f}, CH={ch_score:.1f},
DB={db_score:.3f}')

# Визуализация результатов

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)

plt.plot(K_range, silhouette_scores, 'bo-')

plt.title('Силуэт (чем выше, тем лучше)')

plt.xlabel('Число кластеров K')

plt.ylabel('Score')

```

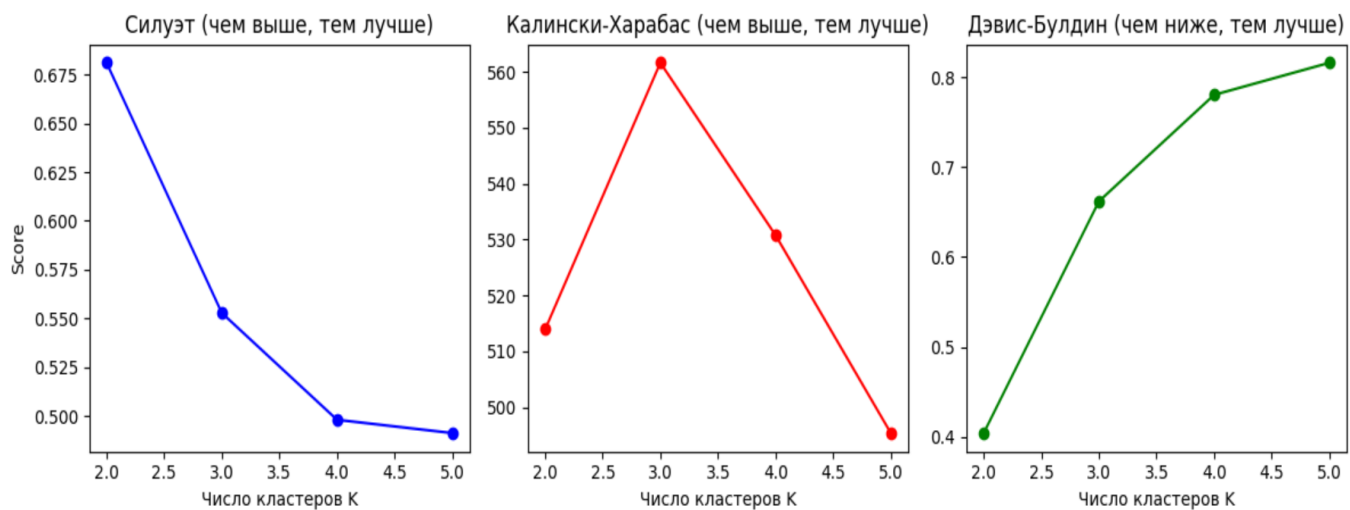
```
plt.subplot(1, 3, 2)
plt.plot(K_range, ch_scores, 'ro-')
plt.title('Калински-Харабас (чем выше, тем лучше)')
plt.xlabel('Число кластеров K')
plt.subplot(1, 3, 3)
plt.plot(K_range, db_scores, 'go-')
plt.title('Дэвис-Булдин (чем ниже, тем лучше)')
plt.xlabel('Число кластеров K')
plt.tight_layout()
plt.show()
```

K=2: Silhouette=0.681, CH=513.9, DB=0.404

K=3: Silhouette=0.553, CH=561.6, DB=0.662

K=4: Silhouette=0.498, CH=530.8, DB=0.780

K=5: Silhouette=0.491, CH=495.4, DB=0.816



Объяснение:

1. silhouette_score (Силуэт):

- Измеряет, насколько точка похожа на свой кластер по сравнению с соседними. Значение от -1 до 1.
- Чем ближе к 1, тем лучше. Пик на графике часто указывает на оптимальное K. В нашем случае K=2 и K=3 дают близкие результаты, но K=3 немного выше.

2. calinski_harabasz_score (CH / Индекс Калински-Харабаша):

- Отношение дисперсии *между* кластерами к дисперсии *внутри* кластеров. Чем больше, тем лучше (кластеры далеко друг от друга и плотные).
- Резкое замедление роста или пик может указывать на хорошее K . Здесь рост замедляется после $K=3$.

3. `davies_bouldin_score` (**DB** / Индекс Дэвиса-Булдина):

- Среднее "похожести" каждого кластера с его самым похожим соседом. Чем меньше, тем лучше.
- Минимум на графике — кандидат в оптимальные K . Здесь минимум достигается при $K=3$.

Вывод по задаче 2: Все три метрики сходятся на том, что оптимальное число кластеров для ирисов — **3** (максимум Силуэта и CH , минимум DB), что соответствует истине, хоть мы ее и не использовали.

Краткий справочник

Метрика	Функция в sklearn	Требует <code>y_true</code> ?	Диапазон	Когда использовать
Скорректированный Rand (ARI)	<code>metrics.adjusted_rand_score</code>	Да	$[-0.5, 1]$	Универсальная метрика, если есть правильные ответы. Сравнение двух разбиений.
V-мера	<code>metrics.v_measure_score</code>	Да	$[0, 1]$	Если нужно понять характер ошибок: неоднородность или неполнота.
Фаулкс-Мэллоуз (FMI)	<code>metrics.fowlkes_mallows_score</code>	Да	$[0, 1]$	Альтернатива ARI, основанная на точности и полноте для пар.
Коэф. Силуэта	<code>metrics.silhouette_score</code>	Нет	$[-1, 1]$	Выбор числа кластеров, оценка разделимости. Работает с любыми расстояниями.
Калински-Харабаш	<code>metrics.calinski_harabasz_score</code>	Нет	$[0, +\infty)$	Быстрая оценка качества, похожа на Силуэт, но быстрее считается. Любит выпуклые кластеры.
Дэвис-Булдин	<code>metrics.davies_bouldin_score</code>	Нет	$[0, +\infty)$	Альтернатива Силуэту. Минимальное значение — лучшее разделение.
Матрица неточностей	<code>cluster.contingency_matrix</code>	Да	-	Детальный анализ перепутывания классов.
Матрица ошибок пар	<code>cluster.pair_confusion_matrix</code>	Да	-	Анализ на уровне пар точек (основа для расчета ARI, FMI).